

Answer:

✓ 1. Remove top rows

✓ 2. Use first row as headers

Question 30:

Question:

You have a data source that contains a column with **case-sensitive data**.

You have a **Power BI semantic model in DirectQuery mode**.

After connecting to the model, you notice **undefined values and errors**.

You need to resolve the issue.

Solution: You implicitly convert the values into the required type.

Does this meet the goal?

Options:

A. Yes

B. No

Answer:

✗ B. No

Solution Explanation:

- **Implicit type conversion** does **not** address case sensitivity issues.
- Power BI, especially in **DirectQuery mode**, relies on the **underlying database engine** for processing queries. Some databases (like SQL Server) **treat values as case-insensitive by default**, while others (like PostgreSQL) are case-sensitive.
- **Case sensitivity issues** can cause **undefined values and errors** when data is queried, especially if the database treats case variations as different values.
- The correct solution would be to **ensure consistent case handling** by either **applying transformations in Power Query** or **modifying the database collation settings** to avoid mismatches.
- Since **implicit conversion does not resolve case sensitivity issues**, this solution **does not meet the goal**.

Question 31:

You have a data source that contains a column with **case-sensitive data**.

You have a **Power BI semantic model in DirectQuery mode**.

After connecting to the model, you notice **undefined values and errors**.

You need to resolve the issue.

Solution: You change the **semantic model mode**.

Does this meet the goal?

Options:

A. Yes

B. No

Answer:

X B. No

Solution Explanation:

- **Changing the semantic model mode (e.g., from DirectQuery to Import mode) does not directly address case sensitivity issues.**
- The issue arises due to **case-sensitive data handling in the data source**, which can cause mismatches when queried in **Power BI**.
- Even if you **switch from DirectQuery to Import mode**, the underlying data issue **remains unresolved**, leading to the same **undefined values and errors**.
- The correct approach would be to **apply transformations in Power Query**, such as **changing the case of text columns (uppercase/lowercase)** or ensuring **consistent collation settings in the database**.
- Since **changing the semantic model mode does not fix case sensitivity problems**, this solution **does not meet the goal**.

Question 32:

You have a data source that contains a column with **case-sensitive data**.

You have a **Power BI semantic model in DirectQuery mode**.

After connecting to the model, you notice **undefined values and errors**.

You need to resolve the issue.

Solution: You **normalize casing** in the **source query** or **Power Query Editor**.

Does this meet the goal?

Options:

- A. Yes
- B. No

Answer:

✓ A. Yes

Solution Explanation:

- The issue arises because **Power BI may treat case-sensitive values differently from the data source**, leading to **errors or undefined values** when querying data.
- By **normalizing casing** (e.g., converting all text values to **uppercase or lowercase**) in either **the source query (SQL)** or **Power Query Editor**, you ensure **consistent data matching**, eliminating undefined values and errors.
- This approach **directly addresses the root cause** of the issue by ensuring that **Power BI queries return consistent results**, regardless of case sensitivity.
- Since **this method effectively resolves the issue**, the solution **meets the goal**.

Question 33:

You have a data source that contains a column with **case-sensitive data**.

You have a **Power BI semantic model in DirectQuery mode**.

After connecting to the model, you notice **undefined values and errors**.

You need to resolve the issue.

Solution: You **add an index key** and **normalize casing** in the **data source**.

Does this meet the goal?

Options:

- A. Yes
- B. No

Answer:

✓ A. Yes

Solution Explanation:

- **Issue:** Power BI may treat **case-sensitive** values inconsistently, leading to **undefined values and errors** when querying data in **DirectQuery mode**.
- **Adding an index key** provides a **unique identifier** for each row, ensuring consistency when retrieving and joining data.
- **Normalizing casing** (e.g., converting text to **uppercase or lowercase**) in the **data source** ensures **case-insensitive** matching and prevents discrepancies.

Answer:

- ✓ **A. CALCULATE**
- ✓ **B. PARALLELPERIOD**
- ✓ **C. DATE[MONTH]**

Question 53:

You plan to create a report that will display sales data from the last year for multiple regions. You need to restrict access to individual rows of the data on a per region-basis by using roles. Which four actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

Actions

Publish the report.	
Assign users to the role.	
Add a filter to the report.	
Create a role definition.	
Import the data to Power BI Desktop.	

Answer Area

Import the data to Power BI Desktop.
Create a role definition.
Assign users to the role.
Publish the report.

You can define roles and rules within Power BI Desktop. When you publish to Power BI, it also publishes the role definitions.

To define security roles, follow these steps.

- 1. Import data into your Power BI Desktop report (Step 1)**
2. From the Modeling tab, select Manage Roles.
- 3. From the Manage roles window, select Create. (Step 2)**
4. Under Roles, provide a name for the role.
5. Under Tables, select the table to which you want to apply a DAX rule.
6. In the Table filter DAX expression box, enter the DAX expressions. This expression returns a value of true or false. For example: [Entity ID] = "Value" (Step 3)